

DATA STRUCTURE

CSA0308

CO 1 AT 2 - PRESENTATION

Guide Name : RAJARAM

Presented by

S. Deepika shree(192521264)



Array

Student Marks



Graph

Navigation



Stack

Browser



Hash Table

Library

SIMATS ENGINEERING

ASSESSMENT TOOL 2 - Technical Presentation

COURSE NAME: Data Structures

COURSE CODE: CSA0308

Register Number	192521264
Name	Deepika shree, S

Assessment Tool Weightage: 30%

COURSE OUTCOME COVERED

CO1: Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL3, BL4)

Questions:4

EACH QUESTIONS CARRIES: 25 MARKS

Total Marks:100

Duration: ONE HOUR

S. No.	List of Questions
1	A student management system needs to store the marks of 1000 students efficiently. Apply the concepts of primitive and non-primitive data structures to select an appropriate structure for storing and managing the data and prepare a technical presentation justifying your choice based on the efficiency of data structure.
2	Choose a suitable linear or non-linear data structure for a navigation system that stores city connections and finds shortest routes between locations. Prepare a technical presentation explaining why it is appropriate.
3	A web browser supports navigating backward through previously visited pages. Recommend a suitable ADT and prepare a technical presentation explaining how it improves performance efficiency.
4	A digital library allows searching books using ISBN numbers quickly among 50,000 records. Select the most suitable data structure and Prepare a technical presentation justifying using time complexity comparison.

Code of Conduct:

I ___ S. Deepika shree (192521264)___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

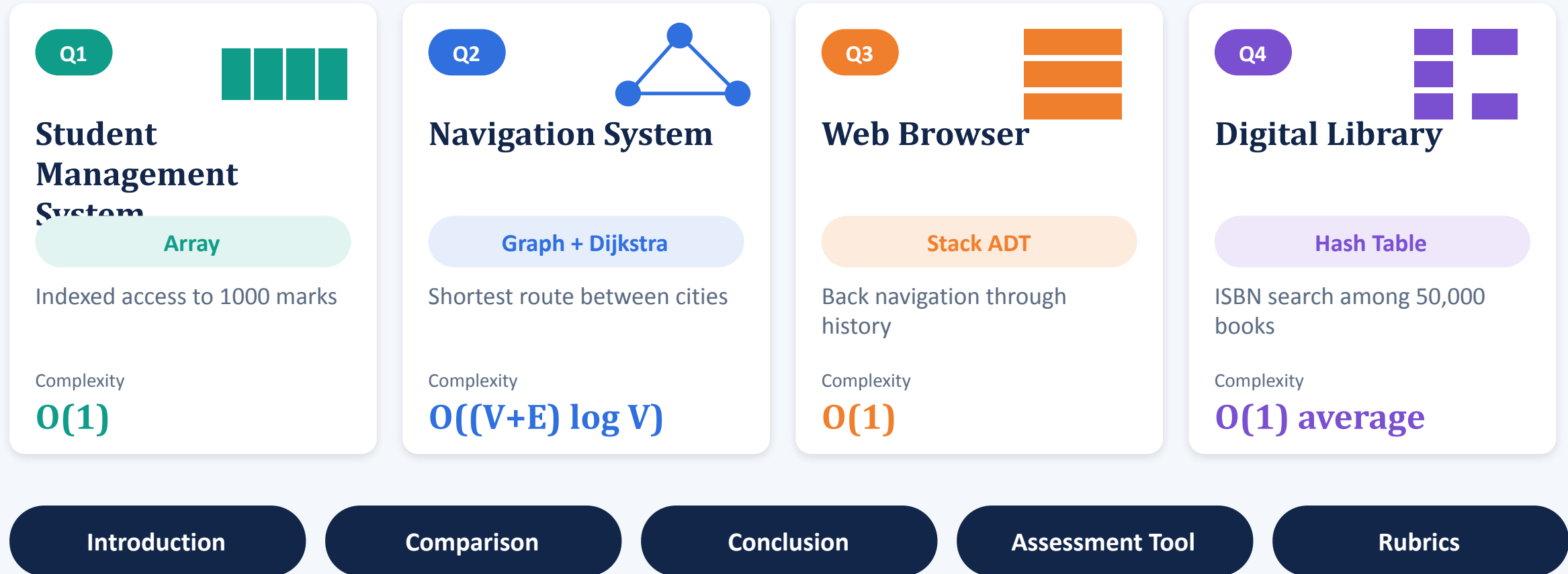
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

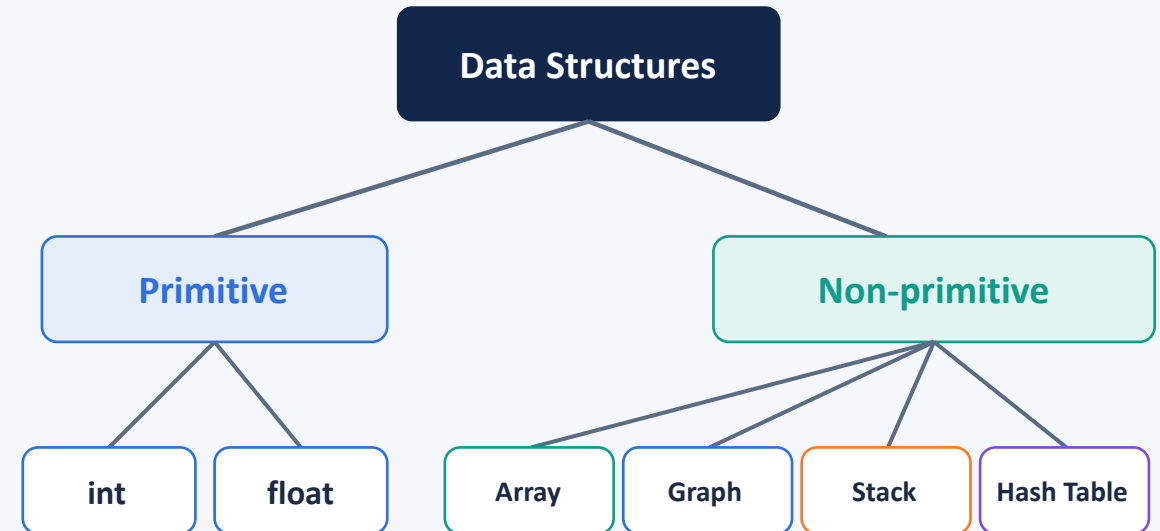
Presentation Roadmap



Introduction



- Data structures are essential for organizing, storing and processing information efficiently.
- Primitive data types store individual values, while non-primitive structures manage collections and relationships.
- Abstract Data Types (ADTs) define operations independently of their internal implementation.
- Selecting an appropriate structure helps reduce execution time and memory usage.
- This presentation examines four applications: student marks management, navigation, browser history and digital library searching.



ADT: operations defined independently of implementation

Stack ADT > array or linked-list implementation

Right structure: less time

Right structure: less memory

Student Management System



Storing Marks of 1000 Students

Problem Analysis and Data Structure Selection

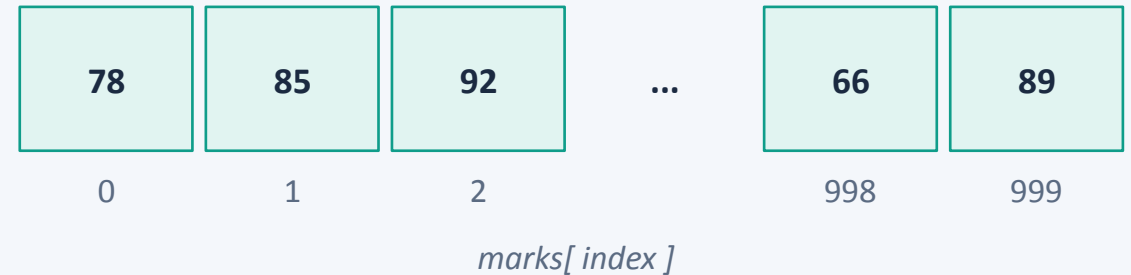
- The system must store and manage the marks of 1000 students efficiently.
- Primitive data types such as int and float represent individual marks.
- Non-primitive data structures organize multiple student marks within a single structure.
- A one-dimensional array is selected because the number of students is known in advance.
- The selected structure should also support accurate mark processing while keeping implementation simple.

Primitive: one value



int / float holds a single mark

Non-primitive: one array for all 1000 marks



marks[1000] = 1000 students

Sample values shown for illustration

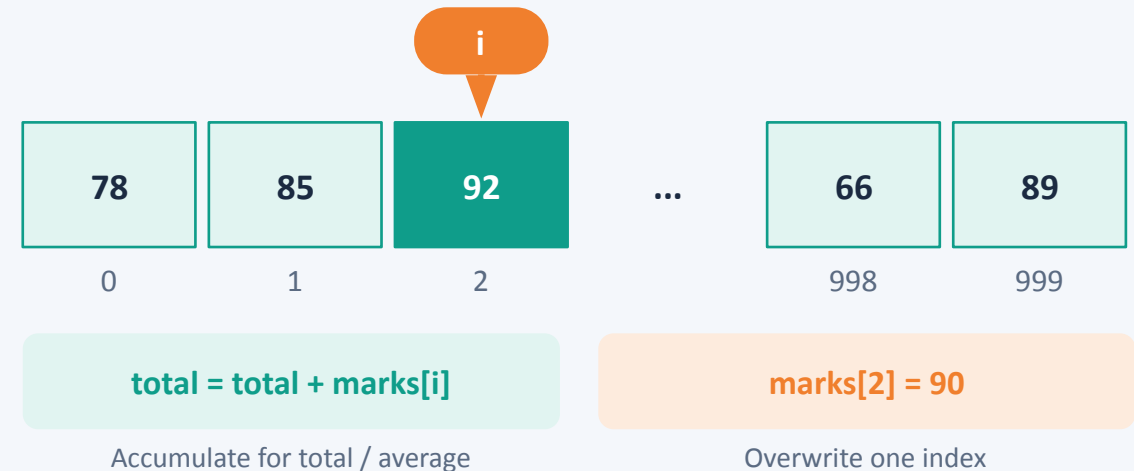
Array Implementation and Operations



- Declare an array using `marks[1000]` to accommodate the marks of all students.
- Assign each student a corresponding array index for retrieving individual marks.
- Use a loop to traverse the array and calculate the total or average marks.
- Modify a student's mark by assigning a new value to the appropriate index.



Traversal with a loop index `i`



Array indices range from 0 to 999, allowing all 1000 student marks to be stored.

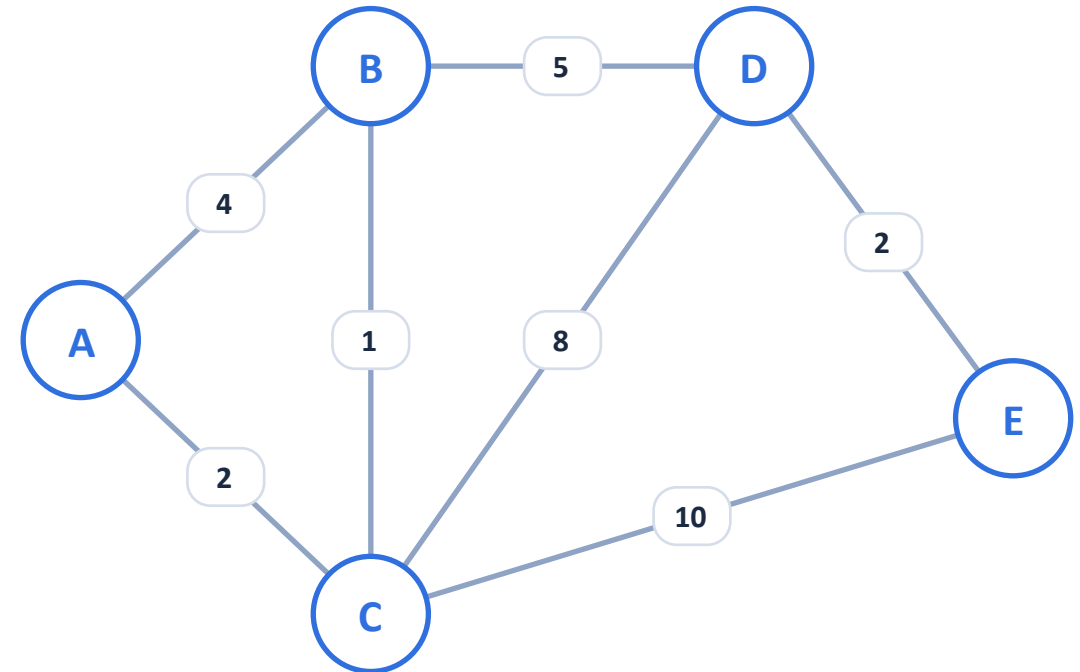
NAVIGATION SYSTEM



City Connections and Shortest Routes

Problem Analysis and Graph Representation

- A navigation system must represent multiple cities and their connecting roads.
- Each city is modelled as a vertex in a graph.
- Each road is modelled as an edge connecting two vertices.
- Edge weights represent road distance or travel time between locations.
- If frequent insertions and deletions are required in the middle, shifting elements can increase the processing time to $O(n)$.



Vertex = city

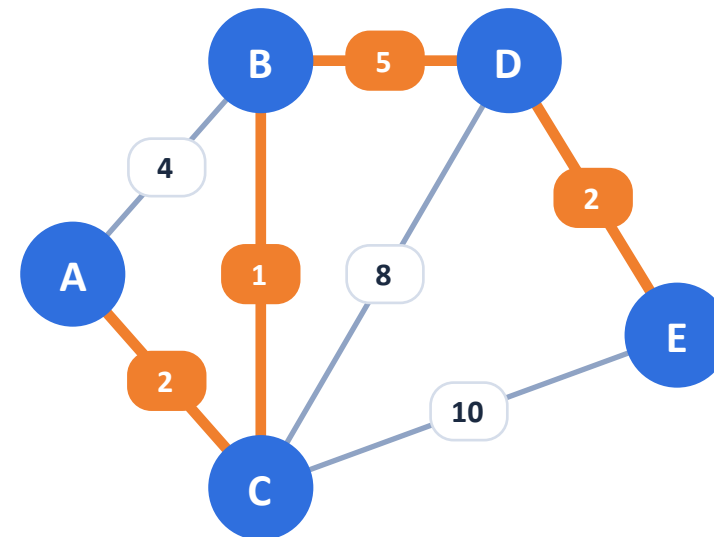
Edge = road

Weight = distance / time

Graph Implementation and Shortest-Path Algorithm



- An adjacency list records the neighbouring cities connected to each location.
- Dijkstra's algorithm identifies the shortest route when all edge weights are non-negative.
- The algorithm maintains the shortest known distance from the starting city.
- After selecting the destination, the stored predecessor information can be used to reconstruct the route.
- An undirected graph can represent two-way roads, while a directed graph can represent one-way roads.



Adjacency list

A: B(4) C(2)

B: A(4) C(1) D(5)

C: A(2) B(1) D(8) E(10)

D: B(5) C(8) E(2)

E: C(10) D(2)

Shortest route A to E: A > C > B > D > E (distance 10)

Shortest known distance from A

A = 0

C = 2

B = 3

D = 8

E = 10

Graph Efficiency and Complexity



- An adjacency list requires $O(V + E)$ space, where V is the number of vertices and E is the number of edges.
- With a binary-heap priority queue, Dijkstra's algorithm takes $O((V + E) \log V)$ time.
- An adjacency list is generally memory-efficient for sparse road networks.
- Dijkstra's algorithm does not directly support negative edge weights; alternative algorithms may be needed for such graphs.

$$O(V + E)$$

Space - adjacency list

$$O((V+E) \log V)$$

Time - Dijkstra with binary heap

Dijkstra loop with a priority queue



WEB BROWSER



Backwards Navigation Using an ADT

Problem Analysis and ADT Selection

- A web browser must maintain the sequence of pages visited during a browsing session.
- The Back button should return the user to the most recently visited previous page.
- A Stack ADT follows the Last-In-First-Out (LIFO) principle.
- A stack is suitable because the latest page can be removed before earlier pages are accessed.
- The stack abstraction separates browser-history operations from the particular method used to store the pages.

https://current-page

Visit page: Push

Back button: Pop

TOP >

Page 3 (current)

Page 2

Page 1

LIFO: Last In, First Out
the latest page leaves first

Stack Operations in Browser History



Push: Add a page to the history when the user navigates to it.

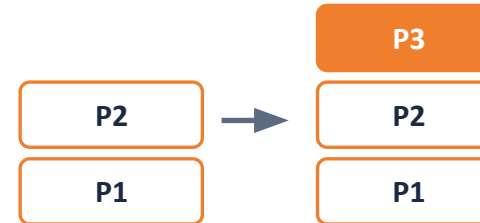
Pop: Remove the current page when the user moves backward.

Peek: Inspect the page at the top without removing it.

Is Empty: Check whether the stack contains any pages before performing a pop operation.

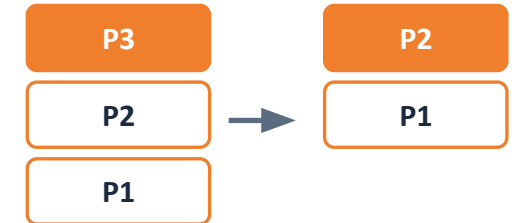
- When a new page is opened after going backward, browser history must manage the forward-history entries appropriately.

Push()



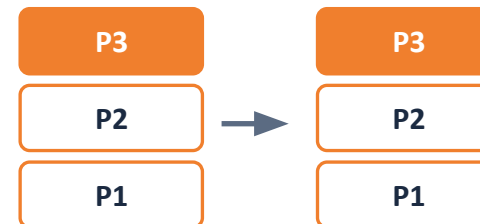
new page on top

Pop()



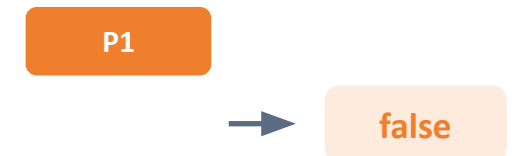
P3 removed

Peek()



reads P3, no change

Is Empty()



false (1 page)

Stack Performance and Limitations



- Push takes $O(1)$ time with a suitable stack implementation.
- Pop takes $O(1)$ time when the stack is non-empty.
- Storing n browsing entries requires $O(n)$ space.
- A complete browser may use separate back and forward stacks to support both navigation directions.
- A linked-list implementation can support dynamic history growth, subject to available memory.

$O(1)$

Push

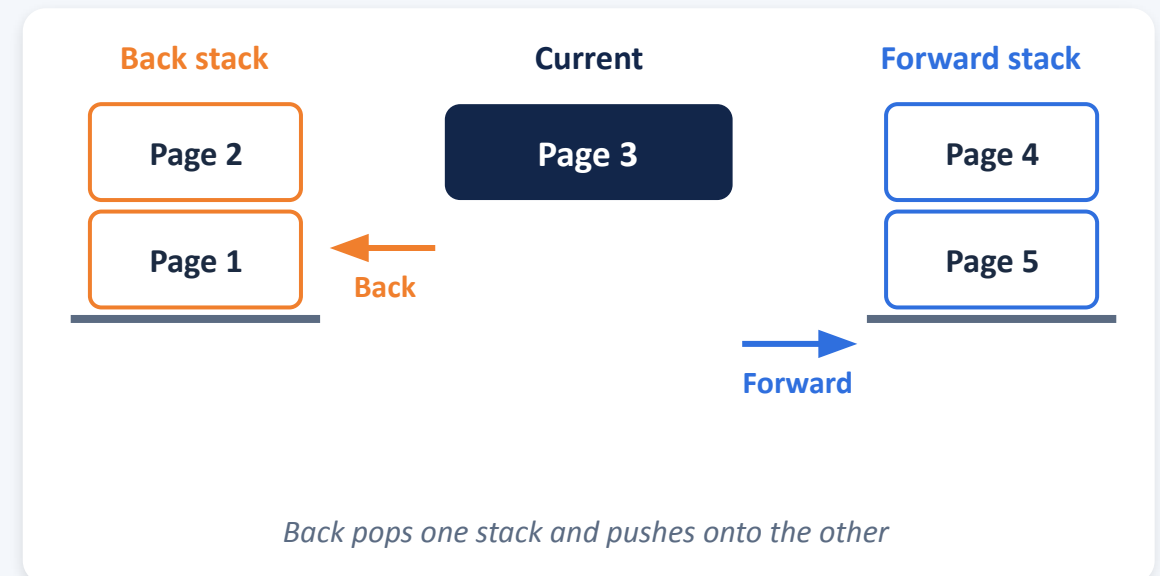
$O(1)$

Pop (non-empty)

$O(n)$

Space for n entries

Back and forward stacks



DIGITAL LIBRARY

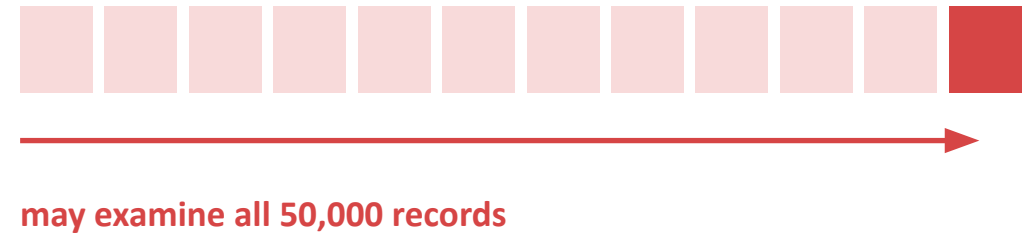


Searching 50,000 Books Using ISBN

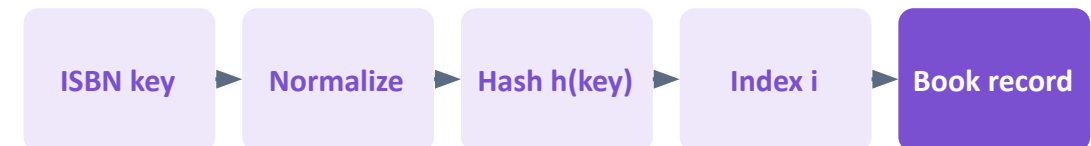
Problem Analysis and Structure Selection

- The digital library contains approximately 50,000 book records.
- Each book's ISBN acts as a key for identifying the required record.
- Sequential searching may examine all 50,000 records when the target is last or absent.
- A hash table is selected to support efficient exact-key searching.
- ISBN values should be normalized into a consistent format before hashing to avoid formatting-related mismatches.

Sequential search



Hash table search

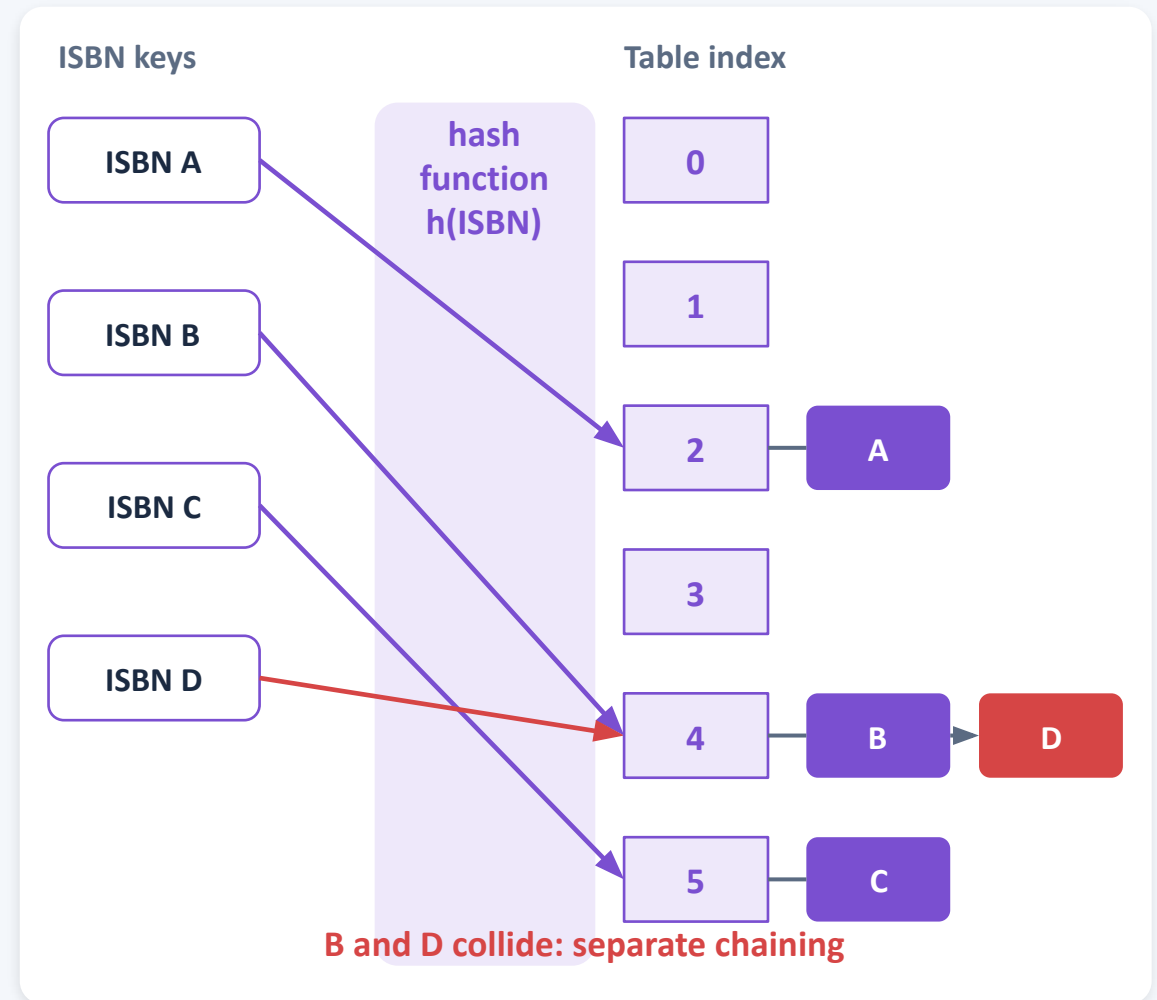


one direct jump to the record

Hash Table Implementation



- Apply a hash function to the ISBN to calculate a table index.
- Store the corresponding book record at the calculated position.
- Use the same hash function to locate the expected position during a search.
- Handle collisions through separate chaining or open addressing when different ISBNs map to the same index.
- A suitable table size and hash function help distribute ISBN keys and reduce collisions.



Hash Table Efficiency and Evaluation



Average search: $O(1)$ under suitable hashing and load conditions.

Average insertion: $O(1)$ under suitable hashing and load conditions.

Worst-case search: $O(n)$ when many keys collide.

Space complexity: $O(n)$ for n stored book records, excluding implementation-specific overhead.

- Search performance should be tested using successful searches, unsuccessful searches and collision-heavy inputs.

$O(1)$

Average search / insertion

$O(n)$

Worst-case search

Record comparisons to find one ISBN ($n = 50,000$)



COMPARISON



Data Structure Comparison

Application	Structure	Main Purpose	Complexity
Student Marks	Array	Indexed Access	$O(1)$
Navigation	Graph + Dijkstra	Shortest Route	$O((V+E) \log V)$
Browser	Stack	Back Navigation	$O(1)$
Library	Hash Table	ISBN Search	$O(1)$ average

*Thank
you!*